

Optimisation du calcul des zéros de ζ : pourquoi le calcul va vite

Auteur : hprzeta — June 1, 2026

Abstract

Ce document explique le **raisonnement mathématique** derrière l'accélération du calcul des zéros non triviaux de la fonction ζ de Riemann sur la droite critique. L'enjeu est de passer d'un débit de l'ordre de 1 zéro/seconde à 15–20 zéros/seconde. L'idée centrale est de séparer rigoureusement deux opérations de nature différente — la *détection* et l'*affinage* — et d'appliquer à chacune l'outil adapté.

1 Introduction : deux opérations distinctes

Localiser un zéro de ζ sur la droite critique se décompose en deux étapes de natures **très différentes** :

1. **Détection** — déterminer qu'un zéro se trouve *quelque part* dans un intervalle, par exemple entre $t = 14.05$ et $t = 14.10$. Opération grossière.
2. **Affinage** — déterminer sa position *exacte*, $t = 14.134725141\dots$, à 12 décimales.

Les deux utilisent la même fonction $Z(t)$, mais **pas de la même façon**. Toute l'optimisation repose sur cette distinction : confondre les deux conduit à utiliser partout l'outil le plus lent et le plus précis, ce qui est inutile pour la détection.

2 La fonction Z de Hardy

On ne cherche pas les zéros de ζ directement : on passe par la fonction Z de Hardy,

$$Z(t) = \Re \left(e^{i\theta(t)} \zeta \left(\frac{1}{2} + it \right) \right),$$

où $\theta(t)$ est la phase de Riemann–Siegel. Sa propriété décisive est que $Z(t)$ **est réelle** pour t réel, avec $|Z(t)| = \left| \zeta \left(\frac{1}{2} + it \right) \right|$. On en déduit l'équivalence fondamentale :

$$\zeta \left(\frac{1}{2} + it \right) = 0 \iff Z(t) = 0.$$

Chercher un zéro de ζ revient ainsi à chercher un **changement de signe** d'une fonction réelle.

3 Étape 1 — Détection : seul le signe compte

On balaie t par petits pas. Un changement de signe de Z entre deux points consécutifs implique, par le théorème des valeurs intermédiaires (Z continue), la présence d'un zéro :

$$Z(14.05) = +1.1, \quad Z(14.10) = -0.3 \implies \exists \gamma \in [14.05, 14.10], \quad Z(\gamma) = 0.$$

Pour cette étape, **aucune haute précision n'est requise** : seul le signe importe. Une évaluation vectorisée `Z_batch()` (NumPy, voire GPU) calcule Z sur tout un tableau de t en une opération, soit un gain d'environ $\times 50$ par rapport à des appels séquentiels de `mpmath.siegelz`.

4 Formule de Riemann–Siegel et nombre de termes

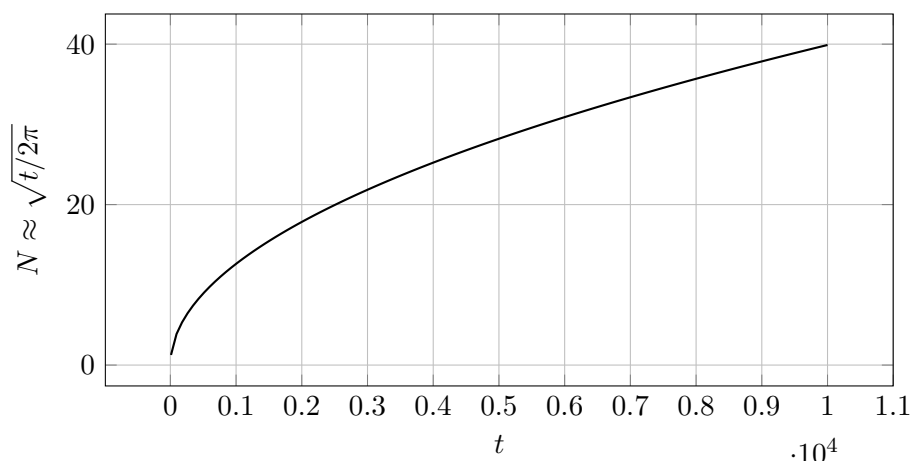
L'évaluation de $Z(t)$ utilise la formule de Riemann–Siegel :

$$Z(t) \approx 2 \sum_{n=1}^N \frac{\cos(\theta(t) - t \log n)}{\sqrt{n}} + R(t), \quad N = \left\lfloor \sqrt{\frac{t}{2\pi}} \right\rfloor.$$

Le point clé : le nombre de termes N **dépend de t** .

t	$N = \lfloor \sqrt{t/2\pi} \rfloor$	Précision interne RS
14	1	imprécise ($\sim 10^{-2}$)
300	6	limite
1000	12	correcte
10000	39	très précise

La croissance de N en $\sqrt{t}$ est illustrée ci-dessous : pour les petits t , la somme comporte trop peu de termes pour être fiable.



5 Seuil d'affinage : $t \geq 300$

L'affinage en C (module `illinois_mpr.so`) recalcule Z via sa propre formule RS interne. Il n'est fiable que lorsque N est suffisant, c'est-à-dire pour $t \geq 300$ ($N \geq 6$). En dessous, on bascule sur `mpmath (fallback)`, ce qui ne concerne que :

$$\#\{\gamma_n : \gamma_n < 300\} \approx 87 \quad \text{sur } 10\,142, \quad \text{soit } < 1\%.$$

On conserve donc la vitesse du C sur 99 % des zéros.

6 Étape 2 — Affinage : la méthode d'Illinois

Un zéro étant encadré dans $[a, b]$ avec $Z(a)Z(b) < 0$, on trace la sécante reliant $(a, Z(a))$ et $(b, Z(b))$; son intersection avec l'axe donne l'estimation :

$$c = b - Z(b) \frac{b - a}{Z(b) - Z(a)}.$$

On conserve le sous-intervalle qui encadre encore le zéro et on itère, resserrant l'encadrement à chaque tour.

Pourquoi « Illinois ». La méthode brute (*regula falsi*) peut voir une borne **stagner**, ralentissant fortement la convergence. La correction d'Illinois : lorsqu'une borne stagne, on divise par 2 la valeur de Z à cette borne, forçant la sécante à pivoter. On obtient une convergence quasi quadratique **tout en garantissant l'encadrement** (aucune divergence possible).

Cas dégénéré. Si la détection place une borne quasi sur le zéro, $|Z_b| \approx 6 \times 10^{-14} \ll |Z_a| \approx 4 \times 10^{-3}$, alors $c \approx b$ et Illinois stagne. La parade : retour immédiat de la borne quasi nulle, et en fin de boucle retour de la meilleure borne (plus petit $|Z|$), jamais le milieu.

7 Parallélisme

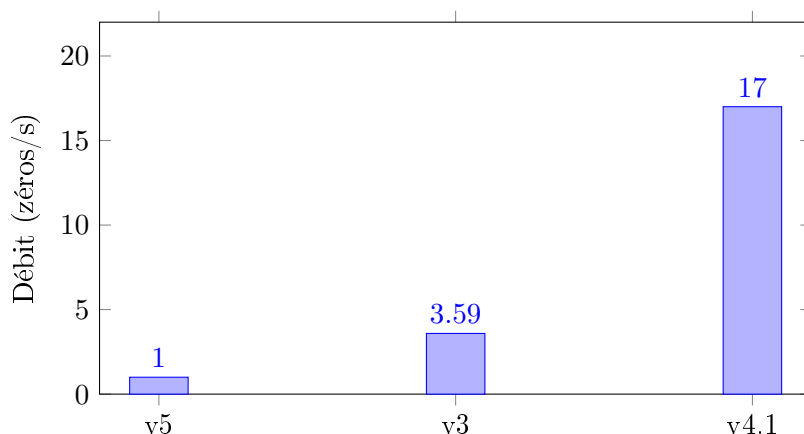
Les intervalles à affiner sont **indépendants** : localiser γ_{500} ne dépend pas de γ_{499} . On répartit donc les intervalles sur les 4 cœurs du processeur, soit un gain d'environ $\times 4$.

Contrainte technique. Le module C `.so` doit être chargé *après* le `fork()`, localement dans chaque *worker* : les objets GMP/MPFR ne se partagent pas proprement à travers un `fork()`.

8 Bilan de l'optimisation

Levier	Agit sur	Gain
<code>Z_batch()</code> vectorisé (NumPy/GPU)	détection	$\sim \times 50$
<code>illinois_mpfr.so</code> (C/libmpfr)	affinage	$\sim \times 39$
4 <i>workers</i> parallèles	ensemble	$\sim \times 4$

Combinés, ces leviers font passer le débit de ~ 1 zéro/s (v5 séquentiel) à ~ 15 – 20 zéros/s (v4.1). Un balayage complet jusqu'à $T = 10\,000$ passe de plusieurs heures à environ 10–15 minutes.



9 Distinction expérimental / preuve

Ces méthodes **localisent numériquement** les zéros non triviaux de $\zeta(s)$ sur $\Re(s) = \frac{1}{2}$. La validation de Turing–Backlund garantit la **complétude numérique** (aucun zéro manqué) sur l'intervalle calculé. Ces résultats ne constituent **pas une preuve** de l'Hypothèse de Riemann.

10 Annexe : code

Détection vectorisée (signe de Z) puis affinage par Illinois sur $t \geq 300$:

```
# Detection : signe de Z sur une grille (vectorise)
t_array = np.arange(t_debut, t_fin, step)
Z_vals  = Z_batch(t_array)                # NumPy / GPU, ~x50
idx      = np.where(np.diff(np.sign(Z_vals)))[0] # changements de signe

# Affinage : Illinois en C si t >= 300, sinon mpmath
for i in idx:
    a, b = t_array[i], t_array[i+1]
    if a >= 300.0:
        gamma = illinois_mpfr(a, b, tol)    # C / libmpfr, ~x39
    else:
        gamma = mpmath.findroot(mpmath.siegelz, (a + b) / 2)
```